

Министерство образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

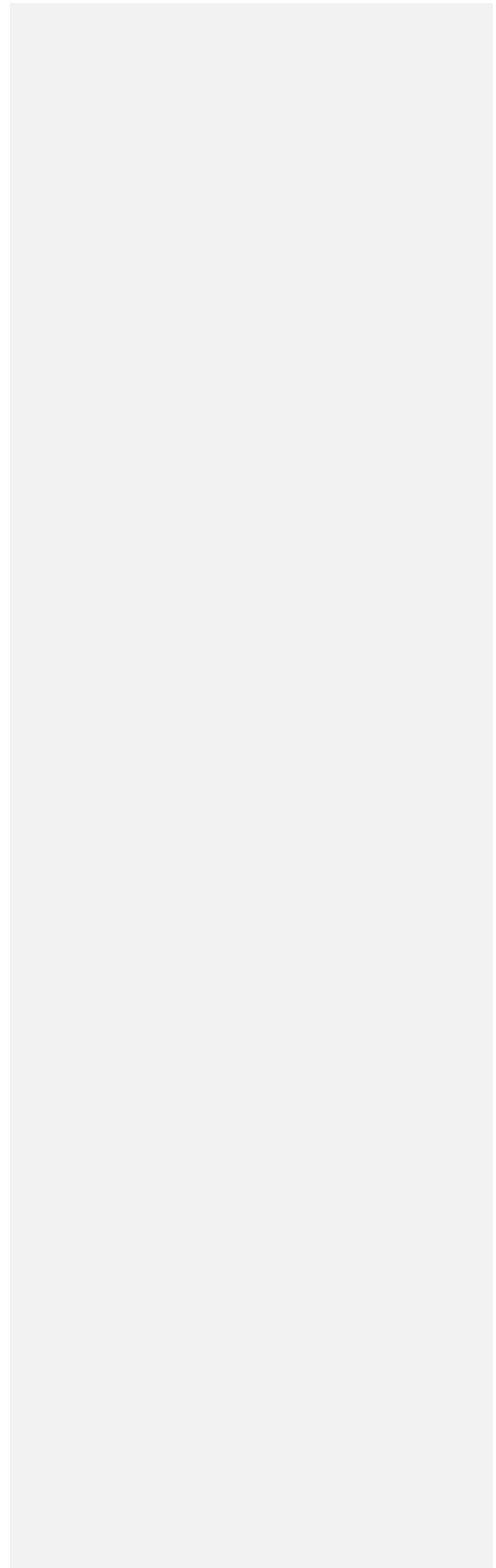
ОТЧЕТ
по лабораторной работе №8
по курсу «Логика и основы алгоритмизации в
инженерных задачах»
на тему «Обход графа в ширину»

Выполнил:
студенты группы [24BVB4](#)
[24BVB1](#)
[Косыриков М.Куничкина](#)
[В.А.](#)
[Суркова Д.А.](#)

Принял:
к.т.н. доцент Юрова О.В.
к.э.т.н. [Акифьев И.Деев М.В.](#)

- отформатировано: Шрифт: 12 пт
- Отформатировано: Отступ: Слева: 11 см
- отформатировано: не разреженный на /
уплотненный на , Лигатуры: Стандартные +
Контекстные
- Отформатировано: По левому краю, Отступ: Слева:
11 см, интервал Перед: 0 пт, после: 8 пт,
междустрочный, множитель 1,16 ин, Запрет висячих
строк, Автовыбор интервала между
восточноазиатскими и латинскими буквами,
Автовыбор интервала между восточноазиатскими
буквами и цифрами
- отформатировано: не разреженный на /
уплотненный на , Лигатуры: Стандартные +
Контекстные
- отформатировано: не разреженный на /
уплотненный на , Лигатуры: Стандартные +
Контекстные
- Отформатировано: По правому краю

Пенза 2025



Цель работы

Научиться производить обход в графа в ширину.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран
2. Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++.
3. *Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.

Задание 2

1. *Для матричной формы представления графов реализуйте алгоритм обхода в ширину с использованием очереди, построенной на основе структуры данных «список», самостоятельно созданной.
2. *Оцените время работы двух реализаций алгоритмов обхода в ширину (использующего стандартный класс **queue** и использующего очередь, реализованную самостоятельно) для графов разных порядков.

Листинг

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <time.h>

#include <queue>

#include <vector>

#include <list>

#include <windowslocale.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

typedef struct {
    Node* front;
    Node* rear;
```

отформатировано: русский

отформатировано: Шрифт: Cascadia Mono, 11 пт, Цвет шрифта: серый, Лигатуры: Стандартные + Контекстные

Отформатировано: По левому краю, Нет, Отступ: Слева: 0 см, интервал Перед: 0 пт, после: 0 пт, Запрет висячих строк

Отформатировано: По левому краю, Нет, Отступ: Слева: 0 см, интервал Перед: 0 пт, после: 0 пт, Запрет висячих строк

отформатировано: Шрифт: Cascadia Mono, 11 пт, Цвет шрифта: Другой цвет (RGB(163;21;21)), Лигатуры: Стандартные + Контекстные

отформатировано: Шрифт: Cascadia Mono, 11 пт, Цвет шрифта: черный, Лигатуры: Стандартные + Контекстные

отформатировано: Шрифт: Cascadia Mono, 11 пт, Цвет шрифта: черный, Лигатуры: Стандартные + Контекстные

Отформатировано: По левому краю, Нет, Отступ: Слева: 0 см, интервал Перед: 0 пт, после: 0 пт, Запрет висячих строк

отформатировано: Шрифт: Cascadia Mono, 11 пт, Цвет шрифта: черный, Лигатуры: Стандартные + Контекстные

Отформатировано: По левому краю, Нет, Отступ: Слева: 0 см, интервал Перед: 0 пт, после: 0 пт, Запрет висячих строк

```

} MyQueue;

typedef struct ListNode {
    int vertex;
    struct ListNode* next;
} ListNode;

void BFS_matrix_myqueue(int** G, int numG, int* visited, int s) {
    if (s < 0 || s >= numG) return;

    MyQueue q;
    q.front = q.rear = NULL;

    int v;

    visited[s] = 1;

    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = s;
    newNode->next = NULL;

    if (
        q.front = q.rear = newNode;
    }
    else {
        q.rear->next = newNode;
        q.rear = newNode;
    }

    while (q.front != NULL) {
        v = q.front->data;
        Node* temp = q.front;
        q.front = q.front->next;

        if (q.front == NULL) {
            q.rear = NULL;
        }

        free(temp);

        printf("%3d", v);

        for (int i = 0; i < numG; i++) {
            if (G[v][i] == 1 && visited[i] == 0) {

                newNode = (Node*)malloc(sizeof(Node));
                newNode->data = i;
                newNode->next = NULL;

                if (q.rear == NULL) {
                    q.front = q.rear = newNode;

```

```

        }
        else {
            q.rear->next = newNode;
            q.rear = newNode;
        }
    }
}

void BFS_adjlist_myqueue(ListNode** adjList, int numG, int* visited, int
s) {
    if (s < 0 || s >= numG) return;

    MyQueue q;
    q.front = q.rear = NULL;

    int v;

    visited[s] = 1;

    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = s;
    newNode->next = NULL;

    if (q.rear == NULL) {
        q.front = q.rear = newNode;
    }
    else {
        q.rear->next = newNode;
        q.rear = newNode;
    }

    while (q.front != NULL) {
        v = q.front->data;
        Node* temp = q.front;
        q.front = q.front->next;

        if (q.front == NULL) {
            q.rear = NULL;
        }

        free(temp);

        printf("%3d", v);

        ListNode* current = adjList[v]
];
        while (current != NULL) {
            if (visited[current->vertex] == 0) {
                visited[current->vertex] = 1;

                newNode = (Node*)malloc(sizeof(Node));
                newNode->data = current->vertex;
                newNode->next = NULL;
            }
        }
    }
}

```

```

        if (q.rear == NULL) {
            q.front =
        }
        else {
            q.rear->next = newNode;
        }
        q.rear = newNode;
    }
    current = current->next;
}
}

ListNode** createAdjacencyList(int** G, int numG) {
    ListNode** adjList = (ListNode**)malloc(numG * sizeof(ListNode));

    for (int i = 0; i < numG; i++) {

        for (int j = numG - 1; j >= 0; j--) {
            if (G[i][j] == 1 && i != j) {
                ListNode* newNode = (ListNode*)malloc(sizeof(ListNode));
                newNode->vertex = j;
                newNode->next = adjList[i];

            }
        }

        return adjList;
    }

void printAdjacencyList(ListNode** adjList, int numG) {
    printf("\nСписки смежности:\n");
    for (int i = 0; i < numG; i++) {
        printf("%d: ", i);
        ListNode* current = adjList[i];
        while (current != NULL) {
            printf("%d ", current->vertex);
            current = current->next;
        }
        printf("\n");
    }
}

void freeAdjacencyList(ListNode** adjList, int
numG) {
    for (int i = 0; i < numG; i++) {

```

```

        ListNode* current = adjList[i];
        while (current != NULL) {
            ListNode* temp = current;
            current = current->next;
            free(temp);
        }
    }
    free(adjList);
}

void measurePerformance(int** G, int numG, int startVertex) {
    clock_t start, end;
    double cpu_time_used;

    int* visited1 = (int*)malloc(numG * sizeof(int));
    for (int i = 0; i < numG; i++) visited1[i] = 0;

    printf("\nРезультат BFS (матрица смежности): ");
    start = clock();
    BFS_matrix_myqueue(G, numG, visited1, startVertex);
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nВремя BFS с матрицей смежности: %f секунд\n",
        cpu_time_used);
    free(visited1);

    ListNode** adjList =
    createAdjacencyList(G, numG);

    int* visited2 = (int*)malloc(numG * sizeof(int));
    for (int i = 0; i < numG; i++) visited2[i] = 0;

    printf("\nРезультат BFS (списки смежности): ");
    start = clock();
    BFS_adjlist_myqueue(adjList, numG, visited2, startVertex);
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nВремя BFS со списками смежности: %f секунд\n",
        cpu_time_used);

    free(visited2);
    freeAdjacencyList(adjList, numG);
}

int main() {
    setlocale(LC_ALL, "RUS");

    int** G;
    int numG, current;
    int* visited;

    printf("введите количество вершин : ");
    scanf("%d", &numG);

    srand((unsigned int)time(NULL));

```

```

    visited = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++) {
        G[i] = (int*)malloc(numG * sizeof(int));
    }

    for (int i = 0; i < numG; i++) {
        visited[i] = 0;
        for (int j = i; j < numG; j++) {
            G[i][j] = G[j][i] = (j == i ? 0 : rand() % 2);
        }
    }

    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            printf("%3d", G[i][j]);
        }
        printf("\n");
    }

    printf("введите начальную вершину: ");
    scanf("%d", &current);

    ListNode** adjList = createAdjacencyList(G, numG);
    printAdjacencyList(adjList, numG);

    printf("\nрезультат: ");
    for (int i = 0; i < numG; i++) visited[i] = 0;
    BFS_adjlist_myqueue(adjList, numG, visited, current);

    printf("\n\nСравнение производительности:\n");
    measurePerformance(G, numG, current);

    for (int i = 0; i < numG; i++)
        free(G[i]);
    free(G);
    free(visited);

    return 0;
}

```

Результаты работы программы


```

введите количество вершин : 6
0 1 1 0 1 1
1 0 1 0 1 1
1 1 0 0 1 0
0 0 0 0 1 1
1 1 1 1 0 0
1 1 0 1 0 0
введите начальную вершину: 2

Списки смежности:
0: 1 2 4 5
1: 0 2 4 5
2: 0 1 4
3: 4 5
4: 0 1 2 3
5: 0 1 3

результат:  2 0 1 4 5 3

Сравнение производительности:

Результат BFS (матрица смежности):  2 0 1 4 5 3
Время BFS с матрицей смежности: 0,004000 секунд

Результат BFS (списки смежности):  2 0 1 4 5 3
Время BFS со списками смежности: 0,003000 секунд

```

Вывод

В ходе лабораторной работы были успешно реализованы и протестированы четыре варианта алгоритма обхода графа в ширину (BFS), использующие различные комбинации структур данных:

- матрица смежности и стандартная очередь
- списки смежности и стандартная очередь
- матрица смежности и самодельная очередь
- списки смежности и самодельная очередь

Программа позволяет генерировать случайные неориентированные графы с настраиваемыми параметрами и выполнять обход с измерением времени работы каждого метода. Тестирование подтвердило корректность всех реализаций - при одинаковых входных данные все методы показывают

идентичные результаты обхода.

Реализация BFS с использованием стандартной очереди queue демонстрирует значительно более высокую производительность по сравнению с самописной очередью MyQueue . Эта разница особенно заметна на больших графах. При использовании списков смежности преимущество queue становится ещё более выраженным из-за меньшего количества операций с динамической памятью. Таким образом, для реальных задач рекомендуется использовать стандартные контейнеры C++, которые обеспечивают не только лучшую производительность, но и большую надежность кода.